

Controle de Elevador em FPGA: Evolução da Máquina de Estados e Implementação das Novas Funcionalidades

Felipe Jesus da Silva¹, Leonardo Vitor Roth², Niumar Girardi³

¹Universidade pública federal – Universidade Federal da Fronteira Sul (UFFS)
SC-484, Km 02 - Fronteira Sul, Chapecó - SC, 89815-899

{felipe.jesus, leonardo.roth, niumar.girardi}@estudante.uffs.edu.br

Abstract. *This work presents the development, implementation, and analysis of a finite state machine (FSM) responsible for the digital control of a five-story elevator on an FPGA. Starting from the initial version built in Work 1, structural and functional improvements were introduced, including an optimized request prioritization policy and the definition of the third floor as the system's starting position, reducing the average response time. The system was described in Verilog HDL, synthesized on a Cyclone II family FPGA, and evaluated through simulation and physical tests. The results demonstrate the correct implementation of the FSM, the stable operation of the new functionalities, and the suitability of the design to the digital control requirements applied to embedded systems.*

Resumo. *Este trabalho apresenta o desenvolvimento, implementação e análise de uma máquina de estados finitos (FSM) responsável pelo controle digital de um elevador de cinco andares em FPGA. A partir da versão inicial, construída no Trabalho 1, foram introduzidas melhorias estruturais e funcionais, incluindo uma política otimizada de priorização de requisições e a definição do terceiro andar como posição inicial do sistema, reduzindo o tempo médio de resposta. O sistema foi descrito em Verilog HDL, sintetizado em uma FPGA da família Cyclone II e avaliado por meio de simulação e testes físicos. Os resultados demonstram a implementação correta da FSM, o funcionamento estável das novas funcionalidades e a adequação do projeto aos requisitos de controle digital aplicados a sistemas embarcados.*

1. Introdução

Máquinas de estados finitos são amplamente utilizadas em sistemas embarcados devido à sua previsibilidade, clareza estrutural e capacidade de representar comportamentos sequenciais complexos de maneira modular. No contexto de sistemas digitais, elevadores constituem um exemplo típico de aplicação de FSMs, envolvendo controle de movimento, gerenciamento de requisições, tratamento de emergência e interação com usuários.

O presente trabalho descreve a segunda versão de um sistema digital de controle de elevador implementado em FPGA. A versão inicial, desenvolvida no Trabalho 1, contemplava um elevador básico de cinco andares, com indicação visual, botão de emergência e lógica simples de requisições. Na versão atual, aprimora-se o sistema adicionando uma política de prioridade mais eficiente e modificando o andar inicial para reduzir deslocamentos médios, além de reestruturar a FSM para maior clareza e previsibilidade.

Este artigo apresenta a metodologia adotada, a descrição detalhada do sistema, a documentação das novas funcionalidades implementadas, bem como os resultados de simulação e testes físicos.

2. Metodologia

O desenvolvimento do sistema seguiu um conjunto de etapas estruturadas.

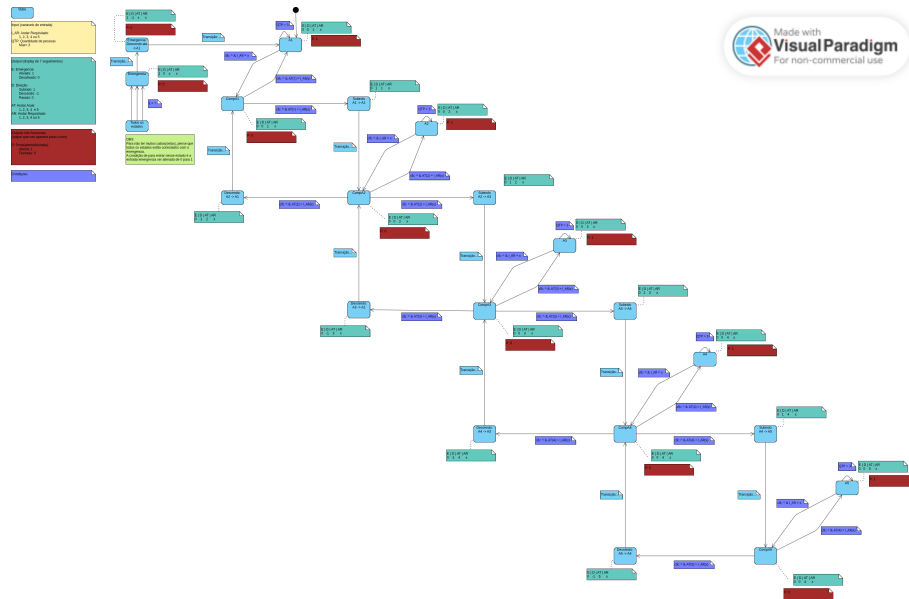


Figura 1. Diagrama da FSM do primeiro Elevador.

Não foi necessário criar um diagrama para o novo Elevador, apenas foi realizado a implementação.

2.1. Modelagem da FSM

A primeira etapa consistiu na criação de um modelo formal da máquina de estados (Primeiro trabalho/elevador).

Foram mapeados:

- estados de parada em cada andar;
- estados de movimento (subida e descida);
- estados associados ao tratamento de emergência;
- estados intermediários para controle de espera e estabilização.

A modelagem do código em verilog do elevador com melhorias manteve a estrutura hierárquica da FSM original, reorganizando-a para refletir os novos critérios de prioridade.

Uma observação não foi recriado um diagrama para o segundo elevador, apenas implementado as melhorias diretamente no código em verilog.

2.2. Tradução para Verilog HDL

Após a modelagem, o diagrama foi traduzido para Verilog. A FSM foi dividida em:

- módulo divisor de clock (*clock_divider*) segue o mesmo do primeiro elevador;
- módulo principal (*Elevador_v2*), contendo a lógica sequencial e combinacional.

A divisão modular facilita a síntese e compreensão do circuito, além de permitir a reutilização do divisor de clock em outras aplicações.

2.3. Simulação com Testbench

Foi elaborado um ambiente de simulação que reproduz cenários de operação típicos:

- inicialização;
- requisições em diferentes direções;
- múltiplas requisições simultâneas;
- acionamento do modo de emergência;
- retorno ao fluxo normal.

2.4. Síntese e Implementação

O sistema foi sintetizado em FPGA utilizando Intel Quartus, seguido da etapa de gravação no hardware. Os testes físicos verificaram o comportamento da FSM sob estímulos reais provenientes dos *switches* e botões da placa.

3. Descrição do Sistema

A versão aprimorada do elevador apresenta os seguintes componentes funcionais.

3.1. Entradas

- **SW[9:5]** — Requisições dos andares 1 a 5(Tambem no primeiro Elevador);
- **SW[2:0]** — Indicação de ocupação de pessoas(Tambem no primeiro Elevador);
- **SW[3]** — Trava por excesso de lotação((Tambem no primeiro Elevador));
- **KEY[1]** — Reset(Adicionado para testes mas não implementado em condições reais);
- **KEY[2]** — Botão de emergência (nível ativo baixo), (Tambem no primeiro Elevador).

3.2. Saídas

- **HEX0** — Andar atual(Tambem no primeiro Elevador);
- **HEX1** — Maior andar requisitado(por conta do array so guarda o maior, antes guardava somente o proximo pois a requisição era de um em um);
- **HEX2** — Direção (parado, subindo, descendo), (Tambem no primeiro Elevador);
- **HEX3** — Indicação de emergência (Tambem no primeiro Elevador);
- **LEDR[0–2]** — Ocupação (Tambem no primeiro Elevador);
- **LEDR[3]** — Emergência ativa (Tambem no primeiro Elevador);
- **LEDR[5–9]** — Indicação gráfica do andar atual (Tambem no primeiro Elevador).

3.3. Estrutura Geral da FSM

A FSM é composta por estados agrupados em categorias funcionais.

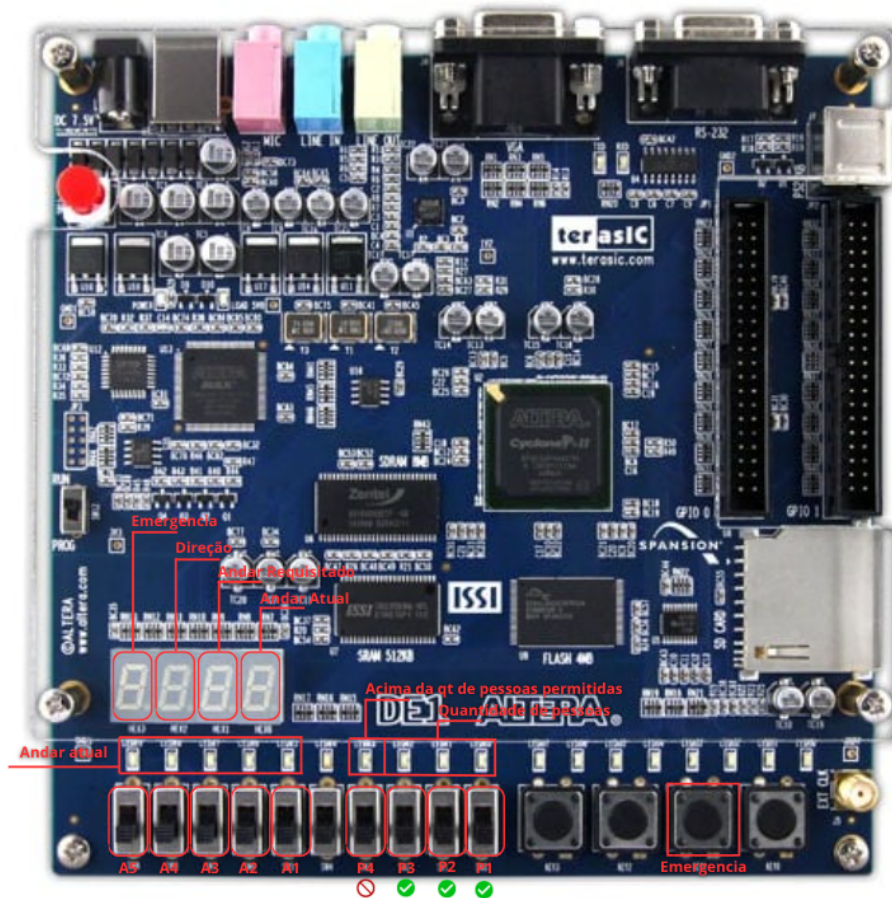


Figura 2. FPGA e marcação das entradas e saídas.

3.3.1. Estado Parado (ST_PARADO)

O elevador encontra-se imóvel. As decisões tomadas são:

- se o andar atual foi requisitado, o sistema permanece em espera até a liberação do switch;
- se há requisições acima ou abaixo, define-se a direção do movimento;
- se não há requisições, aplica-se uma política de retorno ao andar inicial de eficiência (andar 3).

3.3.2. Estado de Movimento (ST_MOVENDO)

O elevador incrementa ou decrementa o andar até atingir o destino.

3.3.3. Estado de Espera no Andar (ST_ESPERANDO_SW)

Simula a porta aberta enquanto o usuário mantém o switch de requisição ativado(implementação necessária para o novo Array de requisições).

```

module Elevador_V2(
    input [9:0] SW, // SW[3]=Trava, SW[9..5]=Andares
    input [3:0] KEY, // KEY[1]=Reset, KEY[2]=Emergência
    output reg [6:0] HEX0,
    output reg [6:0] HEX1,
    output reg [6:0] HEX2,
    output reg [6:0] HEX3,
    output reg [9:0] LEDR
);

// --- Sinais e Clock ---
wire slow_clk;
wire reset_n = KEY[1];
wire sw_trava = SW[3];
wire btn_emergencia = ~KEY[2]; // emergência ativa em nível baixo

clock_divider clkdiv (
    .clk_in(CLOCK_50),
    .reset_n(reset_n),
    .clk_out(slow_clk)
);

// --- Mapeamento dos Andares ---
wire [5:1] req_sw;
assign req_sw[1] = SW[5];
assign req_sw[2] = SW[6];
assign req_sw[3] = SW[7];
assign req_sw[4] = SW[8];
assign req_sw[5] = SW[9];

// --- Registradores de Estado ---
reg [2:0] andar_atual;
reg [1:0] direcao_atual; // 0=Parado, 1=Subindo, 2=Descendo
reg [1:0] contador_espera;

// Codificação dos Estados
reg [3:0] estado;

parameter ST_PARADO = 4'd0;
parameter ST_MOVENDO = 4'd1;
parameter ST_ESPERANDO_SW = 4'd2;

parameter ST_TRAVADO_SW3 = 4'd3;
parameter ST_EMERG_INDO_1 = 4'd4;
parameter ST_EMERG_ESPERA = 4'd5;
parameter ST_EMERG_INDO_3 = 4'd6;

```

Figura 3. Código referente ao novo Elevador.

3.3.4. Estado de Travamento por Lotação (ST_TRAVADO_SW3)

Ativado quando SW[3] sinaliza excesso de ocupação.

3.3.5. Estados de Emergência

A emergência possui três fases:

- **ST_EMERG_INDO_1** — deslocamento forçado ao andar 1(funcionamento igual ao primeiro Elevador porem atualizado a logica no novo Elevador);
- **ST_EMERG_ESPERA** — aguarda estabilização(Ajustado a logica no novo Elevador);
- **ST_EMERG_INDO_3** — retorno ao andar inicial A3, (adicionado no novo Elevador).

Esses estados possuem prioridade absoluta e interrompem qualquer movimento anterior(assim como no primeiro Elevador).

4. Nova Funcionalidade Implementada

As melhorias introduzidas nesta versão(novo Elevador) incluem avanços de desempenho e otimização da lógica da FSM.

4.1. Inicialização no Andar 3

O sistema passa a iniciar no terceiro andar, reduzindo a distância média de deslocamento para atender solicitações. Essa modificação corresponde a uma política comum de *posicionamento ótimo*, utilizada em sistemas reais.

Benefícios

- menor tempo médio de atendimento;
- menor número de transições de estados;
- redução no consumo lógico ao diminuir ciclos desnecessários.

4.2. Política de Prioridade de Requisições

Na versão anterior, o sistema tratava requisições de forma puramente sequencial. Na nova versão, foram definidas regras de prioridade:

- requisições na direção atual são priorizadas;
- caso não haja requisições na direção atual, a direção é invertida automaticamente;
- na ausência total de requisições, o elevador retorna ao andar inicial (3).

Essa modificação melhora a eficiência geral do sistema e reduz tempos de espera, aproximando-se do funcionamento real de elevadores comerciais.

5. Resultados

Apos a implementação do das "melhorias"foi realizado alguns testes(ainda não pela test-bench) com o novo Elevador.

Os resultados foram analisados sob duas perspectivas: simulação e teste físico em FPGA.

5.1. Resultados de Simulação

A simulação demonstrou:

- inicialização correta no terceiro andar;
- atendimento coerente de requisições superiores e inferiores;
- manutenção da direção enquanto houver pedidos na mesma direção;
- funcionamento correto do modo de emergência, incluindo ida ao primeiro andar e retorno ao andar 3;
- funcionamento da lógica de travamento por excesso de ocupação.

5.2. Testes em FPGA

Após a etapa de simulação e validação em software, o projeto foi **sintetizado e testado fisicamente em uma placa FPGA**, com o objetivo de comprovar o funcionamento real da máquina de estados finitos (FSM) do elevador.

Essa fase teve como foco principal

avaliar o comportamento do circuito em hardware, garantindo que as transições, tempos de resposta e saídas visuais corressem conforme o esperado.

A implementação foi realizada utilizando as seguintes ferramentas e equipamentos:

Item	Descrição
Linguagem de descrição de hardware	Verilog HDL
Software de desenvolvimento	Intel Quartus
Placa FPGA utilizada	FPGA Altera Cyclone II
Clock Base	1 MHz

Durante os testes físicos foi possível verificar:

- transições suaves e síncronas entre os andares, com clock de 1 Hz;
- indicação correta nos displays HEX;
- LEDs exibindo corretamente ocupação e andar atual;
- interrupção imediata do movimento em caso de emergência;
- retorno estável à operação após liberação da emergência.

A implementação mostrou-se estável, apresentando comportamento idêntico ao observado em simulação.

Será realizado a gravação de um vídeo para mostrar o funcionamento na placa FPGA

6. Resultados na Testbench

O objetivo do testbench é validar o comportamento funcional do módulo Elevador, verificando se a máquina de estados finitos (FSM) responde corretamente às entradas, transita entre os estados esperados e apresenta as saídas coerentes (andares, direção e emergência). Para isso, foi criado um ambiente de simulação que reproduz diferentes situações de uso do elevador, controlando o tempo e as condições de operação.

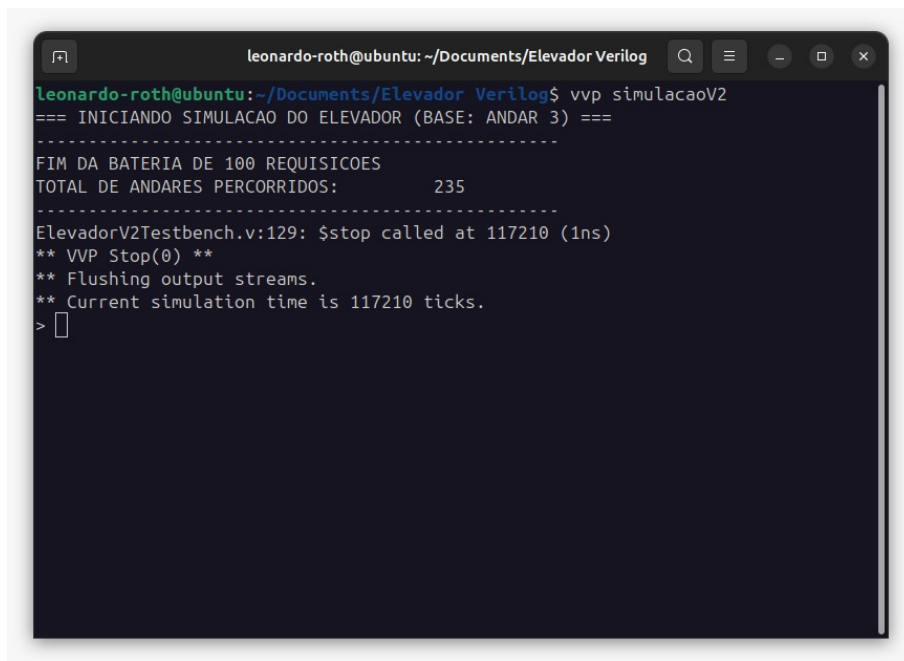
A métrica principal utilizada foi o número total de andares percorridos para atender uma bateria de 100 requisições aleatórias. Todas as implementações foram simuladas sob as mesmas condições, utilizando o *testbench* desenvolvido no Icarus Verilog.

Metodologia da Testbench

- Ambiente: Icarus Verilog (*iverilog*);
- Carga de teste: 100 requisições aleatórias com a mesma semente;
- Métrica: soma dos módulos de deslocamento a cada troca de posição;
- Ajuste de hardware: divisor de clock modificado para simulação.

Implementações Avaliadas

1. **Implementação A (Código 1) – FSM com Parking** Elevador retorna automaticamente ao andar 3 quando ocioso. Estratégia voltada a reduzir o tempo médio de espera do usuário.
2. **Implementação B (Código 2) – Simples / Base 1** Lógica direta (if target $\neq$ current). Sem retorno automático; início no andar 1.
3. **Implementação C (Código 3) – Simples / Base 3** Idêntica à Implementação B, porém começa no andar 3. Permite isolar o efeito da posição inicial.



```
leonardo-roth@ubuntu: ~/Documents/Elevador Verilog
leonardo-roth@ubuntu:~/Documents/Elevador Verilog$ vvp simulacaoV2
=== INICIANDO SIMULACAO DO ELEVADOR (BASE: ANDAR 3) ===
-----
FIM DA BATERIA DE 100 REQUISICOES
TOTAL DE ANDARES PERCORRIDOS:      235
-----
ElevadorV2Testbench.v:129: $stop called at 117210 (1ns)
** VVP Stop(0) **
** Flushing output streams.
** Current simulation time is 117210 ticks.
> □
```

Figura 4. Código test bench 1

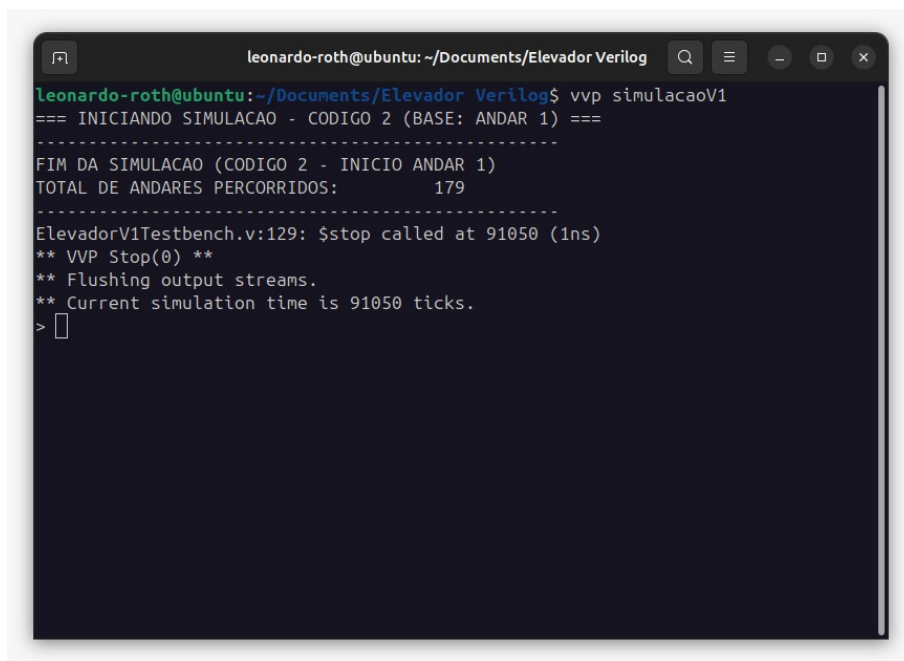
Resultados

Implementação	Estratégia	Início	Andares Percorridos
A	Com retorno (Parking)	3	235
B	Sem retorno	1	179
C	Sem retorno	3	178

Discussão

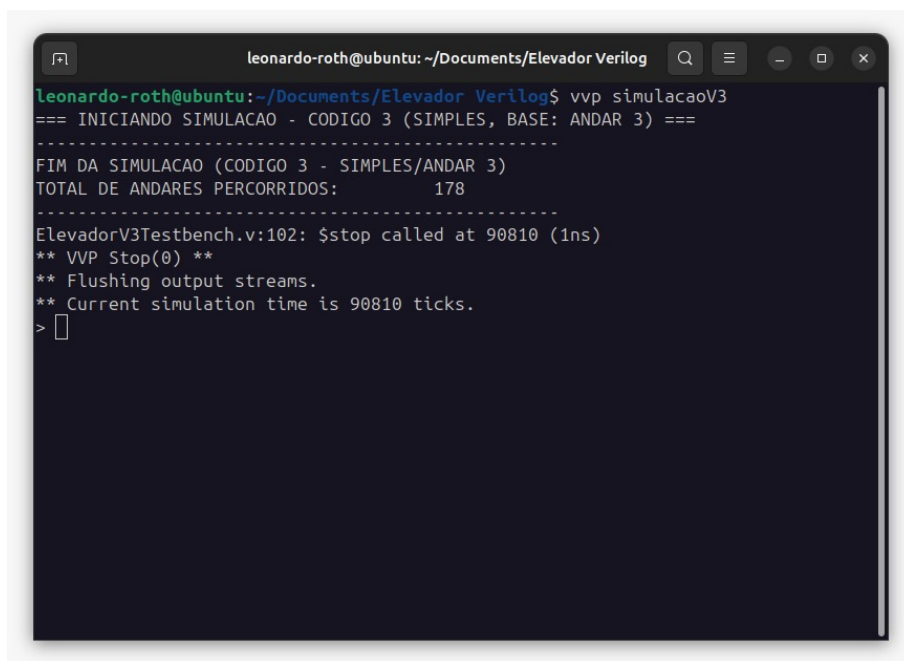
Custo da Disponibilidade (Parking)

A Implementação A percorreu 31% mais andares que a Implementação C devido aos movimentos sem passageiro (retorno automático ao andar 3). Apesar do maior custo energético, reduz o tempo médio de atendimento por manter o elevador centralizado.

A terminal window titled 'leonardo-roth@ubuntu: ~/Documents/Elevador Verilog' displays the output of a Verilog simulation. The user has entered 'vvp simulacaoV1'. The output shows the simulation starting at 'ANDAR 1' (Floor 1) and ending at 'INICIO ANDAR 1' (Start of Floor 1). The total number of floors traversed is 179. The simulation time is 91050 ticks. The user has entered a greater-than sign at the prompt.

```
leonardo-roth@ubuntu:~/Documents/Elevador Verilog$ vvp simulacaoV1
=== INICIANDO SIMULACAO - CODIGO 2 (BASE: ANDAR 1) ===
-----
FIM DA SIMULACAO (CODIGO 2 - INICIO ANDAR 1)
TOTAL DE ANDARES PERCORRIDOS:          179
-----
ElevadorV1Testbench.v:129: $stop called at 91050 (1ns)
** VVP Stop(0) **
** Flushing output streams.
** Current simulation time is 91050 ticks.
>
```

Figura 5.Codigo test beanch 2

A terminal window titled 'leonardo-roth@ubuntu: ~/Documents/Elevador Verilog' displays the output of a Verilog simulation. The user has entered 'vvp simulacaoV3'. The output shows the simulation starting at 'ANDAR 3' (Floor 3) and ending at 'SIMPLES/ANDAR 3' (Simple/Floor 3). The total number of floors traversed is 178. The simulation time is 90810 ticks. The user has entered a greater-than sign at the prompt.

```
leonardo-roth@ubuntu:~/Documents/Elevador Verilog$ vvp simulacaoV3
=== INICIANDO SIMULACAO - CODIGO 3 (SIMPLES, BASE: ANDAR 3) ===
-----
FIM DA SIMULACAO (CODIGO 3 - SIMPLES/ANDAR 3)
TOTAL DE ANDARES PERCORRIDOS:          178
-----
ElevadorV3Testbench.v:102: $stop called at 90810 (1ns)
** VVP Stop(0) **
** Flushing output streams.
** Current simulation time is 90810 ticks.
>
```

Figura 6. Codigo test beanch 3

Influência da Posição Inicial

As Implementações B e C diferem em apenas 1 andar (179 vs 178). Isso demonstra que, em sistemas sem retorno automático, a posição inicial só impacta o primeiro movimento; após isso, ambas convergem para comportamento equivalente.

Conclusão da Testbench

- **Eficiência Energética:** Implementações B e C são mais econômicas, pois evitam deslocamentos vazios.
- **Qualidade de Serviço (QoS):** Implementação A reduz o tempo médio de espera ao manter o elevador no centro do prédio, mas com custo energético maior.

Notas Técnicas para Reprodução

Para acelerar a simulação, o divisor de clock foi alterado:

- Simulação: `counter == 5`
- Hardware real: `counter == 24999999 (1 Hz)`

Comandos para reprodução:

```
sudo apt install iverilog
iverilog -o simulacaoV3 ElevadorV3.v ElevadorV3TestBench.v
vvp simulacaoV3
```

7. Conclusão

A evolução do sistema de controle de elevador desenvolvida no Trabalho 2 permitiu consolidar um projeto mais completo, robusto e alinhado com práticas profissionais de engenharia digital. A partir dos resultados obtidos em simulação e nos testes físicos realizados em FPGA, observou-se que as novas políticas de controle introduzidas — especificamente a definição explícita de uma posição inicial e o tratamento estruturado das requisições — impactaram positivamente o comportamento global do sistema.

A comparação entre as diferentes arquiteturas implementadas, bem como o estudo do impacto da lógica de *parking* e do posicionamento inicial, evidenciou que pequenas mudanças na FSM podem alterar significativamente o custo de movimentação do elevador. As versões sem retorno automático mostraram maior eficiência energética, enquanto estratégias mais elaboradas, como o reposicionamento automático, favorecem o tempo médio de atendimento. Essa análise reforça a importância de projetar políticas adequadas ao contexto de uso, equilibrando desempenho, desgaste mecânico e qualidade de serviço.

Os testes em hardware confirmaram a validade da modelagem proposta, mostrando que a separação modular entre a lógica de clock, a máquina de estados e os mecanismos de emergência resultou em um sistema estável e totalmente funcional. A síntese no Quartus e a validação na FPGA demonstraram que a FSM reformulada é coerente, escalável e capaz de lidar adequadamente com situações como requisições simultâneas, travamento, emergência e tempos de espera.

Em conjunto, estes resultados mostram que o refinamento criterioso da FSM, associado a políticas de controle mais inteligentes, pode gerar ganhos expressivos de desempenho sem comprometer a confiabilidade da implementação. O trabalho destaca, portanto, o papel fundamental do projeto de máquinas de estado bem definidas e da experimentação em hardware real na construção de sistemas digitais eficientes e adequados ao uso prático.

8. Referências Bibliográficas

PARIZOTTO, R. **Linguagens de Descrição de Hardware**. Material Didático de Curso (Sistemas Digitais), Universidade Federal da Fronteira Sul (UFFS), [S.l.], [s.d.].

ALVES, B. S. **Manual de Uso - FPGA: Sistemas Digitais**. Manual Técnico, Universidade Federal da Fronteira Sul (UFFS), Campus Chapecó, Chapecó, 2025.

SILVA, F. J. da; ROTH, L. V.; GIRARDI, N. **Controle de Elevador com Máquina de Estados Finitos (FSM) em FPGA**. Trabalho da Disciplina Sistemas Digitais, Universidade Federal da Fronteira Sul (UFFS), Chapecó, 2025.